一. Ceres Solver 库

- **官网：**

  http://ceres-solver.org/

- **安装：**

  sudo apt-get install liblapack-dev libsuitesparse-dev libcxsparse3.1.4 libgflags-dev libgoogle-glog-dev libgtest-dev

  git clone https://github.com/ceres-solver/ceres-solver.git

  cd ceres-solver/

  mkdir build

  cd build

  cmake ..

  make

  make install

- **Ceres 求解最小二乘问题：**

  Ceres 库面向通用的最小二乘问题的求解，用户需要做的就是定义优化问题，然后设置一些选项，输入进 Ceres 求解即可。

  步骤一般包含：定义代价函数计算模型；构建最小二乘问题；配置求解器。

$$\begin{aligned} \min_{x} \quad & \frac{1}{2}\sum_{i}\rho_i\left(\left\|f_i\left(x_{i_1},\ldots x_{i_n}\right)\right\|^2\right) \\ s.t. \quad & l_j \le x_j \le u_j. \end{aligned} \tag{6.27}$$

可以看到，目标函数由许多平方项，经过一个**核函数** $\rho(\cdot)$ 之后，求和组成[①]。在最简单的情况下，取 ρ 为恒等函数，则目标函数即为许多项的平方和。在这个问题中，优化变量为 $x_1,\ldots,x_n$，f_i 称为**代价函数**（Cost function），在 SLAM 中亦可理解为误差项。l_j 和 u_j 为第 j 个优化变量的上限和下限。在最简单的情况下，取 $l_j=-\infty, u_j=\infty$（不限制优化变量的边界），并且取 ρ 为恒等函数时，就得到了无约束的最小二乘问题，和我们先前说的是一致的。

二. 拟合曲线

与机器学习回归问题类似：《回归问题：线性回归》

完整代码：https://github.com/deng-ke/slam-base/tree/master/ceres_curve_fitting。

- **目标：**

  使用 Ceres 拟合曲线：

$$y = \exp(ax^2 + bx + c) + \omega$$

- **输入输出：**

输入：N 对 x、y；高斯噪声 ω。

输出：曲线参数 a、b、c。

- **方法**:

  求如下最小二乘问题以估计曲线参数。（相当于机器学习中最小化代价函数）

$$\min_{a,b,c}\frac{1}{2}\sum_{i=1}^{N}\|y_i-\exp(ax_i{}^2+bx_i+c)\|^2$$

2.1 定义代价函数计算模型

以仿函数形式，重载()，定义代价函数。

```
/*****************************************************************************
 *定义代价函数的计算模型
 *****************************************************************************/
struct CURVE_FITTING_COST
{
    CURVE_FITTING_COST ( double x, double y ) : _x ( x ), _y ( y ) {}

    /*残差的计算（重载()，仿函数）*/
    template <typename T>
    bool operator() (
        const T* const abc,   // 模型参数，有 3 维
        T* residual ) const   // 残差
    {
        residual[0] = T ( _y ) - ceres::exp ( abc[0]*T ( _x ) *T ( _x ) + abc[1]*T ( _x ) + abc[2] );
        return true;
    }

    /*x、y 数据*/
    const double _x, _y;
};
```

2.2 生成数据

根据真实的模型生成 x、y 真值，再加入随机的高斯噪声。

```
/*****************************************************************************
 *生成数据
 *****************************************************************************/
    double a=1.0, b=2.0, c=1.0;             // 真实参数值
    int N=100;                                  // 数据点
```

```
    double w_sigma=1.0;                 // 噪声 Sigma 值
    cv::RNG rng;                        // OpenCV 随机数产生器
    double abc[3] = {0,0,0};            // abc 参数的估计值
    vector<double> x_data, y_data;      // 数据

    /*根据真实的模型生成 x、y 真值，再加入随机的高斯噪声*/
    cout<<"generating data: "<<endl;
    for ( int i=0; i<N; i++ )
    {
        double x = i/100.0;
        x_data.push_back ( x );
        y_data.push_back (
            exp ( a*x*x + b*x + c ) + rng.gaussian ( w_sigma )
        );
        cout<<x_data[i]<<" "<<y_data[i]<<endl;
    }
```

2.3 构建最小二乘问题

导入前面定义的代价函数模型。

```
/*****************************************************************************
 *构建最小二乘问题
 *****************************************************************************/
    ceres::Problem problem;
    for ( int i=0; i<N; i++ )
    {
        problem.AddResidualBlock (      // 向问题中添加误差项
        // 使用自动求导，模板参数：误差类型，输出维度，输入维度。（维数要与前面 struct 中一致）
            new ceres::AutoDiffCostFunction<CURVE_FITTING_COST, 1, 3> (
                new CURVE_FITTING_COST ( x_data[i], y_data[i] )
            ),
            nullptr,                // 核函数，这里不使用，为空
            abc                     // 待估计参数
        );
    }
```

2.4 配置求解器

```
/*****************************************************************************
 *配置求解器
```

```
**********************************************************************************/
    ceres::Solver::Options options;        // 这里有很多配置项可以填
    options.linear_solver_type = ceres::DENSE_QR;   // 增量方程如何求解
    options.minimizer_progress_to_stdout = true;     // 输出到 cout

    ceres::Solver::Summary summary;                  // 优化信息
    chrono::steady_clock::time_point t1 = chrono::steady_clock::now();
    ceres::Solve ( options, &problem, &summary );   // 开始优化
    chrono::steady_clock::time_point t2 = chrono::steady_clock::now();
    chrono::duration<double>                         time_used                    =
chrono::duration_cast<chrono::duration<double>>( t2-t1 );
    cout<<"solve time cost = "<<time_used.count()<<" seconds. "<<endl;

    // 输出结果
    cout<<summary.BriefReport() <<endl;
    cout<<"estimated a,b,c = ";
    for ( auto a:abc ) cout<<a<<" ";
    cout<<endl;
```

2.5 结果

```
iter      cost      cost_change  |gradient|   |step|    tr_ratio  tr_radius  ls_iter  iter_time  total_time
   0  1.824887e+04    0.00e+00    1.38e+03   0.00e+00   0.00e+00  1.00e+04        0    3.98e-05    1.48e-04
   1  2.748700e+39   -2.75e+39    0.00e+00   7.67e+01  -1.52e+35  5.00e+03        1    5.29e-05    3.17e-04
   2  2.429783e+39   -2.43e+39    0.00e+00   7.62e+01  -1.35e+35  1.25e+03        1    2.41e-05    3.96e-04
   3  1.213227e+39   -1.21e+39    0.00e+00   7.30e+01  -6.73e+34  1.56e+02        1    2.31e-05    5.10e-04
   4  1.852387e+37   -1.85e+37    0.00e+00   5.56e+01  -1.03e+33  9.77e+00        1    2.22e-05    5.86e-04
   5  6.714689e+31   -6.71e+31    0.00e+00   2.96e+01  -3.85e+27  3.05e-01        1    2.41e-05    6.65e-04
   6  9.500531e+12   -9.50e+12    0.00e+00   9.50e+00  -8.39e+08  4.77e-03        1    2.10e-05    7.38e-04
   7  1.776982e+04    4.79e+02    1.83e+03   2.58e-01   1.18e+00  1.43e-02        1    4.79e-05    8.37e-04
   8  1.599969e+04    1.77e+03    3.45e+03   5.53e-01   1.46e+00  4.29e-02        1    4.41e-05    9.33e-04
   9  1.060557e+04    5.39e+03    7.62e+03   7.33e-01   1.68e+00  1.29e-01        1    4.29e-05    1.03e-03
  10  3.669783e+03    6.94e+03    9.60e+03   5.25e-01   1.39e+00  3.86e-01        1    4.20e-05    1.12e-03
  11  5.397541e+02    3.13e+03    5.00e+03   2.66e-01   1.12e+00  1.16e+00        1    4.20e-05    1.21e-03
  12  1.484444e+02    3.91e+02    1.22e+03   8.46e-02   1.02e+00  3.48e+00        1    4.20e-05    1.30e-03
  13  1.216815e+02    2.68e+01    3.76e+02   4.17e-02   1.01e+00  1.04e+01        1    4.20e-05    1.40e-03
  14  9.290109e+01    2.88e+01    2.42e+02   9.10e-02   1.01e+00  3.13e+01        1    4.79e-05    1.52e-03
  15  6.674330e+01    2.62e+01    1.09e+02   1.33e-01   1.00e+00  9.39e+01        1    5.51e-05    1.65e-03
  16  5.936574e+01    7.38e+00    2.14e+01   1.08e-01   9.94e-01  2.82e+02        1    4.41e-05    1.75e-03
  17  5.653118e+01    2.83e+00    1.36e+01   1.57e-01   9.98e-01  8.45e+02        1    4.29e-05    1.84e-03
  18  5.310764e+01    3.42e+00    8.50e+00   2.81e-01   9.89e-01  2.53e+03        1    4.20e-05    1.94e-03
  19  5.125939e+01    1.85e+00    2.84e+00   2.98e-01   9.90e-01  7.60e+03        1    4.29e-05    2.03e-03
  20  5.097693e+01    2.82e-01    4.34e-01   1.48e-01   9.95e-01  2.28e+04        1    4.20e-05    2.12e-03
  21  5.096854e+01    8.39e-03    3.24e-02   2.87e-02   9.96e-01  6.84e+04        1    4.20e-05    2.22e-03
solve time cost = 0.00230917 seconds.
Ceres Solver Report: Iterations: 22, Initial cost: 1.824887e+04, Final cost: 5.096854e+01, Termination: CONVERGENCE
estimated a,b,c = 0.891943 2.17039 0.944142
```